بسم تعالی

ساختمان داده ها و طراحی الگوریتم ها
پروژه پایانی

دانشجویان گرامی، ضمن آرزوی موفقیت

لطفا با دقت موارد زیر را مطالعه کرده و در پیاده‌سازی پروژه رعایت فرمائید.

کدهای نوشته شده به همراه تمامی موارد استفاده شده را در یک فایل zip به صورت StudentNumber_FinalProject.zip (به عنوان مثال 40131062_FinalProject.zip) قرار داده و در سامانه کورسز در بازه زمانی تعیین شده ارسال نمایید.

پروژه را می‌توان با استفاده از زبان برنامه‌نویسی دلخواه نوشت فقط دقت کنید اجازه استفاده از هیچ کتابخانه خارجی وجود ندارد.

در صورت نیاز به راهنمایی یا بروز هر گونه مشکلی می توانید از طریق تلگرام یا بله با تدریسیاران درس در ارتباط باشید.

آخرین مهلت ارسال پروژه تا ساعت ۲۳:۵۹ یکشنبه ۲۸ تیر می باشد.

هرگونه شباهت در کل یا بخشی از پاسخنامه‌ها فارغ از اینکه از چه منبعی تهیه شده باشند، به عنوان عدم رعایت حق کپی رایت لحاظ خواهد شد و نمره بین تمامی نسخه های مشابه تقسیم خواهد شد.

کلیه دانشجویان مجازند در خصوص تحلیل پروژه، مطالعه مستندات، تبادل ایده با ابزارهای هوش مصنوعی و چت‌بات‌ها یا با یکدیگر همکاری نمایند؛ اما طراحی نهایی، پیاده‌سازی کد پروژه باید به‌صورت انفرادی انجام شده و توسط خود دانشجو تحویل گردد

موفق باشید

مقدمه

در سال‌های اخیر، سیستم‌های In-Memory Database نقش بسیار مهمی در توسعه‌ی نرم‌افزارهای مقیاس‌پذیر و با کارایی بالا ایفا کرده‌اند. یکی از شناخته‌شده‌ترین نمونه‌های این سیستم‌ها، Redis است که به دلیل سرعت بالا، سادگی رابط کاربری و پشتیبانی از انواع مختلف ساختارهای داده، در بسیاری از سامانه‌های بزرگ دنیا مورد استفاده قرار می‌گیرد.

هدف این پروژه، طراحی و پیاده‌سازی نسخه‌ای ساده‌شده از Redis است؛ به گونه‌ای که دانشجویان تمامی ساختارهای داده و الگوریتم‌های موردنیاز را از ابتدا پیاده‌سازی کنند و از هیچ کتابخانه‌ی آماده برای ساختارهای داده استفاده نکنند.



قوانین مربوط به استفاده از کتابخانه‌ها و زبان برنامه‌نویسی

هدف اصلی این پروژه، آشنایی عمیق با نحوه‌ی طراحی و پیاده‌سازی ساختمان داده‌ها و الگوریتم‌ها در پایین‌ترین سطح ممکن است. به همین دلیل، تمامی اجزای پروژه باید **از صفر** پیاده‌سازی شوند و استفاده از هرگونه پیاده‌سازی آماده برای ساختمان داده‌ها و الگوریتم‌ها اکیداً ممنوع است. در این پروژه، استفاده از هر زبان برنامه‌نویسی مجاز است، اما استفاده از هرگونه ساختمان داده و الگوریتم آماده در آن زبان، حتی موارد پایه‌ای مانند لیست‌ها، آرایه‌های پویا، مجموعه‌ها، دیکشنری‌ها، صف‌ها، پشته‌ها، مرتب‌سازی‌های آماده و توابع جستجو، به‌طور کامل ممنوع است و تمامی این موارد باید توسط دانشجو از ابتدا پیاده‌سازی شوند.

همچنین استفاده از کتابخانه‌ها و توابع آماده تنها در حد مدیریت ورودی و خروجی و مدیریت حافظه مجاز است. به عبارت دیگر، دانشجویان صرفاً می‌توانند از توابعی برای خواندن ورودی، چاپ خروجی، تخصیص و آزادسازی حافظه استفاده کنند و استفاده از هرگونه تابع یا کتابخانه‌ی دیگر برای پردازش داده‌ها، رشته‌ها یا پیاده‌سازی ساختمان داده‌ها و الگوریتم‌ها مجاز نیست.

شرح پروژه

در این پروژه، هدف شما طراحی و پیاده‌سازی یک نسخه‌ی ساده‌شده از Redis است. تمامی ساختارهای داده و الگوریتم‌های موردنیاز باید از ابتدا پیاده‌سازی شوند و استفاده از هرگونه کتابخانه‌ی آماده برای پیاده‌سازی ساختارهای داده مجاز نیست.

پروژه باید به‌صورت یک برنامه‌ی خط فرمان پیاده‌سازی شود که دستورات را از ورودی استاندارد دریافت کرده، آن‌ها را پردازش نموده و خروجی مناسب را در خروجی استاندارد نمایش دهد.

در ادامه، دستورات موردنیاز پروژه به ترتیب معرفی شده‌اند.

بخش اول: دستورات پایه

۱- دستور SET:

```text
SET <key> <value>
```

در صورتی که کلید وجود نداشته باشد، یک کلید جدید ایجاد کرده و مقدار داده‌شده را در آن ذخیره می‌کند. اگر کلید از قبل وجود داشته باشد، مقدار قبلی با مقدار جدید جایگزین می‌شود.

پیچیدگی زمانی مورد انتظار:

در حالت میانگین باید در زمان $O(1)$ انجام شود.

نمونه ورودی:

```text
SET name Armin
SET age 22
```

۲- دستور GET:

```text
GET <key>
```

مقدار متناظر با کلید موردنظر را برمی‌گرداند. در صورتی که کلید وجود نداشته باشد، پیام مناسبی (مانند `NULL` یا `Key Not Found`) نمایش داده شود.

پیچیدگی زمانی مورد انتظار:

در حالت میانگین باید در زمان $O(1)$ انجام شود.

نمونه ورودی:

```text
GET name
```

نمونه خروجی:

```text
Armin
```

۳- دستور DEL:

```text
DEL <key>
```



کلید و مقدار متناظر با آن را از پایگاه داده حذف می‌کند. در صورتی که کلید وجود نداشته باشد، برنامه نباید دچار خطا شود.

پیچیدگی زمانی مورد انتظار:

در حالت میانگین باید در زمان $O(1)$ انجام شود.

نمونه ورودی:

```text
DEL age
```

۴- دستور EXISTS:

```text
EXISTS <key>
```

بررسی می‌کند که آیا کلید موردنظر در پایگاه داده وجود دارد یا خیر.

خروجی:

- 1 در صورت وجود کلید
- 0 در غیر این صورت

پیچیدگی زمانی مورد انتظار:

در حالت میانگین باید در زمان $O(1)$ انجام شود.

نمونه ورودی:

```text
EXISTS name
EXISTS score
```

نمونه خروجی:

```text
1
0
```

بخش دوم: دستورات زمان انقضا

در این بخش، سیستم باید قابلیت تعیین زمان انقضا برای کلیدها را پشتیبانی کند. به این معنا که هر کلید می‌تواند یک مدت‌زمان مشخص برای زنده ماندن داشته باشد و پس از پایان این مدت، به‌صورت خودکار نامعتبر شده و از پایگاه داده حذف شود. این قابلیت باید برای تمامی کلیدها، از جمله کلیدهایی که در دستورات بعدی نیز ایجاد یا استفاده می‌شوند، به‌صورت یکپارچه اعمال شود.

۵- دستور EXPIRE:

```text
EXPIRE <key> <seconds>
```

برای کلید موردنظر، زمان انقضا تعیین می‌کند. پس از گذشت تعداد ثانیه‌های مشخص‌شده، کلید باید از پایگاه داده حذف شود یا در صورت دسترسی بعدی، نامعتبر در نظر گرفته شود.

اگر کلید وجود نداشته باشد، این دستور نباید باعث ایجاد کلید جدید شود.

پیچیدگی زمانی مورد انتظار:

باید در زمان $O(1)$ انجام شود.

نمونه ورودی:

```text
SET token abc123
EXPIRE token 60
```

۶- دستور TTL:

```text
TTL <key>
```

مدت‌زمان باقی‌مانده تا انقضای کلید را بر حسب ثانیه برمی‌گرداند.

پیچیدگی زمانی مورد انتظار:

باید در زمان $O(1)$ انجام شود.



خروجی پیشنهادی:

- اگر کلید وجود نداشته باشد، مقدار 2- برگردانده شود.
- اگر کلید وجود داشته باشد ولی زمان انقضا نداشته باشد، مقدار 1- برگردانده شود.
- اگر کلید دارای زمان انقضا باشد، تعداد ثانیه‌های باقی‌مانده برگردانده شود.

نمونه ورودی:

```text
TTL token
```

نمونه خروجی:

```text
45
```

یعنی ۴۵ ثانیه تا انقضای کلید `token` باقی مانده است.



بخش سوم: دستورات لیست

در این بخش، سیستم باید از نوع داده‌ی List پشتیبانی کند. لیست در این پروژه دنباله‌ای مرتب از مقادیر رشته‌ای است که امکان اضافه‌کردن و حذف‌کردن عنصر از ابتدا و انتهای آن وجود دارد.

۷- دستور LPUSH:

```text
LPUSH <key> <value>
```

مقدار داده‌شده را به ابتدای لیست اضافه می‌کند. اگر کلید موردنظر وجود نداشته باشد، یک لیست جدید ساخته می‌شود.

پیچیدگی زمانی مورد انتظار:

این عملیات باید در زمان $O(1)$ انجام شود.

نمونه ورودی:

```text
LPUSH numbers 10
LPUSH numbers 20
```

پس از اجرای دستورات بالا، لیست `numbers` به شکل زیر خواهد بود:

```text
20 10
```

۸- دستور RPUSH:

```text
RPUSH <key> <value>
```

مقدار داده‌شده را به انتهای لیست اضافه می‌کند. اگر کلید موردنظر وجود نداشته باشد، یک لیست جدید ساخته می‌شود.



پیچیدگی زمانی مورد انتظار:

این عملیات باید در زمان $O(1)$ انجام شود.

نمونه ورودی:

```text
RPUSH numbers 30
```

پس از اجرای دستورات بالا، لیست `numbers` به شکل زیر خواهد بود:

```text
20 10 30
```

۹- دستور LPOP:

```text
LPOP <key>
```

اولین عنصر لیست را حذف کرده و همان مقدار را به‌عنوان خروجی نمایش می‌دهد. اگر لیست خالی باشد یا کلید وجود نداشته باشد، پیام مناسبی مانند `NULL` نمایش داده شود.

پیچیدگی زمانی مورد انتظار:

این عملیات باید در زمان $O(1)$ انجام شود.

نمونه ورودی:

```text
LPOP numbers
```

نمونه خروجی:

```text
20
```



۱۰- دستور RPOP:

```text
RPOP <key>
```

آخرین عنصر لیست را حذف کرده و همان مقدار را به‌عنوان خروجی نمایش می‌دهد. اگر لیست خالی باشد یا کلید وجود نداشته باشد، پیام مناسبی مانند NULL نمایش داده شود.

پیچیدگی زمانی مورد انتظار:

این عملیات باید در زمان $O(1)$ انجام شود.

۱۱- دستور LRANGE:

```text
LRANGE <key> <start> <end>
```

عناصر لیست را از اندیس `start` تا اندیس `end` نمایش می‌دهد. اندیس‌ها از صفر شروع می‌شوند و هر دو کران `start` و `end` در خروجی لحاظ می‌شوند.

پیچیدگی زمانی مورد انتظار:

در حالت کلی، این عملیات باید در زمان $O(n)$ انجام شود.

نمونه ورودی:

```text
LRANGE numbers 0 2
```

نمونه خروجی:

```text
20 10 30
```





بخش چهارم: دستورات مجموعه‌ی مرتب

۱۲- دستور ZADD:

```text
ZADD <key> <score> <member>
```

عضو `member` را با امتیاز `score` به مجموعه‌ی مرتب `key` اضافه می‌کند. اگر مجموعه‌ی موردنظر وجود نداشته باشد، ابتدا یک مجموعه‌ی مرتب جدید ساخته می‌شود.

اگر عضو از قبل در مجموعه وجود داشته باشد، امتیاز قبلی آن با امتیاز جدید جایگزین شده و جایگاه عضو در ساختار مرتب به‌روزرسانی می‌شود.

خروجی:

- `1` اگر عضو جدید اضافه شود
- `0` اگر عضو از قبل وجود داشته و فقط امتیاز آن به‌روزرسانی شود

پیچیدگی زمانی مورد انتظار:

این عملیات باید در زمان $O(\log n)$ انجام شود.

نمونه ورودی:

```text
ZADD leaderboard 120 Ali
ZADD leaderboard 90 Sara
ZADD leaderboard 150 Reza
```

پس از اجرای دستورات بالا، اعضا بر اساس امتیاز به‌صورت صعودی مرتب می‌شوند:

```text
Sara 90
Ali 120
Reza 150
```

۱۳- دستور ZREM:

```text
ZREM <key> <member>
```

عضو موردنظر را از مجموعه‌ی مرتب حذف می‌کند. اگر مجموعه یا عضو موردنظر وجود نداشته باشد، برنامه نباید دچار خطا شود.

خروجی:

- 1 اگر عضو با موفقیت حذف شود
- 0 اگر عضو وجود نداشته باشد

پیچیدگی زمانی مورد انتظار:

این عملیات باید در زمان $O(\log n)$ انجام شود.

نمونه ورودی:

```text
ZREM leaderboard Sara
```

۱۴- دستور ZSCORE:

```text
ZSCORE <key> <member>
```

امتیاز عضو موردنظر را در مجموعه‌ی مرتب نمایش می‌دهد.

خروجی:

اگر عضو وجود داشته باشد، امتیاز آن نمایش داده شود. در غیر این صورت، خروجی مناسب مانند NULL نمایش داده شود.

پیچیدگی زمانی مورد انتظار:

این عملیات باید در حالت میانگین در زمان $O(1)$ انجام شود.

نمونه ورودی:

```
ZSCORE leaderboard Ali
```

نمونه خروجی:

```
120
```

۱۵- دستور ZRANK:

```
ZRANK <key> <member>
```

رتبه‌ی عضو موردنظر را در مجموعه‌ی مرتب بر اساس ترتیب صعودی امتیاز برمی‌گرداند.

پیچیدگی زمانی مورد انتظار:

این عملیات باید در زمان $O(\log n)$ انجام شود.

نمونه:

اگر مجموعه به شکل زیر باشد:

```
Sara 90
Ali 120
Reza 150
```

ورودی:

```
ZRANK leaderboard Ali
```

خروجی:

```
1
```



بخش پنجم: ساختار داده‌ی گراف

در این بخش، سیستم باید از نوع داده‌ی Graph پشتیبانی کند. هر گراف مجموعه‌ای از رأس‌ها و یال‌ها است که برای مدل‌سازی ساختارهایی مانند شبکه‌ها، نقشه‌ها و ارتباطات مورد استفاده قرار می‌گیرد. در این پروژه، هر گراف با یک نام در پایگاه داده ذخیره می‌شود و امکان ایجاد، افزودن اجزا و اجرای الگوریتم‌های مختلف بر روی آن فراهم خواهد بود.

۱۶- دستور GCREATE:

```text
GCREATE <graph_name>
```

یک گراف جدید و خالی ایجاد می‌کند.

پیچیدگی زمانی مورد انتظار:

این عملیات باید در زمان $O(1)$ انجام شود.

نمونه ورودی:

```text
GCREATE roads
```

۱۷- دستور GADDNODE:

```text
GADDNODE <graph_name> <node>
```

یک رأس جدید به گراف اضافه می‌کند.

اگر رأس از قبل وجود داشته باشد، نباید دوباره ایجاد شود.

پیچیدگی زمانی مورد انتظار:

این عملیات باید در زمان $O(1)$ (یا سرشکن یا میانگین) انجام شود.

نمونه ورودی:

```text
GADDNODE roads Tehran
GADDNODE roads Karaj
```

۱۸- دستور GADDEDGE:

```text
GADDEDGE <graph_name> <source> <destination> <weight>
```

یک یال وزن‌دار بین دو رأس ایجاد می‌کند.

در این پروژه، گراف بدون جهت در نظر گرفته می‌شود؛ بنابراین با اضافه شدن یک یال، ارتباط در هر دو جهت برقرار خواهد شد.

پیچیدگی زمانی مورد انتظار:

این عملیات باید در زمان $O(1)$ (یا سرشکن یا میانگین) انجام شود.

نمونه ورودی:

```text
GADDEDGE roads Tehran Karaj 35
GADDEDGE roads Karaj Qazvin 110
GADDEDGE roads Tehran Qom 150
```

۱۹- دستور GCONNECTED:

```text
GCONNECTED <graph_name> <source> <destination>
```

بررسی می‌کند که آیا بین دو رأس مسیر وجود دارد یا خیر.

خروجی:

- 1 اگر مسیر وجود داشته باشد.
- 0 در غیر این صورت.



پیچیدگی زمانی مورد انتظار:

این عملیات باید در زمان $O(\log n)$ (یا سرشکن یا میانگین) انجام شود.

۲۰- دستور GMST (امتیازی):

```text
GMST <graph_name>
```

درخت پوشای کمینه‌ی گراف را محاسبه کرده و مجموع وزن یال‌های انتخاب‌شده را نمایش می‌دهد.

پیچیدگی زمانی مورد انتظار:

این عملیات باید در زمان $O(\log n)$ سرشکن انجام شود.

راهنمایی پیاده‌سازی:

برای رسیدن به این پیچیدگی، می‌توانید از ساختار داده‌ی Link-Cut Tree استفاده کنید.